

Experiment No.	6
Experiment Title	Dimensionality Reduction and Model Evaluation (With and Without PCA)
GitHub Repository:	https://github.com/KJayasuriya/ml_laboratory/
Student Name	K. Jayasuriya
Register Number	3122247001026
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 30, 2026

1 Objective

The objective of this experiment is to study the effect of Principal Component Analysis (PCA) on the performance of different machine learning classification algorithms.

The experiment compares two settings:

1. Classification using the original feature space without PCA.
2. Classification using a PCA-reduced feature space.

For both settings, multiple classification algorithms are trained using hyperparameter tuning and 5-fold stratified cross-validation. Their performance is then compared using accuracy and F1-score.

The experiment also investigates whether dimensionality reduction improves classification performance, reduces model complexity, or affects different types of classifiers differently.

2 Problem Statement

The task is to build and evaluate multiple machine learning classification models on the given dataset and study the effect of dimensionality reduction using Principal Component Analysis.

The following ten algorithms are evaluated:

1. Support Vector Machine (SVM)
2. Naïve Bayes
3. k-Nearest Neighbors (KNN)
4. Logistic Regression
5. Decision Tree
6. Random Forest
7. AdaBoost
8. Gradient Boosting
9. XGBoost

10. Stacking Ensemble

Each model is evaluated both with and without PCA. Hyperparameter tuning is performed using GridSearchCV with 5-fold stratified cross-validation.

3 Dataset Description

The dataset used in this experiment is a binary classification dataset containing diagnostic measurements.

Dataset Name	Breast Cancer Wisconsin (Diagnostic) Dataset
Dataset Source	UCI Machine Learning Repository
Number of Samples	569
Number of Features	30
Number of Classes	2
Target Classes	Benign (B) and Malignant (M)
Missing Values	No missing values were present in the working dataset
Train-Test Split	80% training and 20% testing

The target variable is the diagnosis class. For model training, the categorical target was encoded numerically as:

$$B = 0, \quad M = 1$$

The dataset contains 30 numerical predictor variables representing characteristics computed from digitized images of breast mass samples.

4 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed before model training to understand the structure and characteristics of the dataset.

The following aspects were examined:

- Dataset dimensions and feature types.
- Target-class distribution.
- Missing-value information.
- Descriptive statistics.
- Feature distributions.
- Feature correlations.
- Distribution of numerical variables.

The dataset contains 569 observations and 30 numerical features. The target variable consists of two classes, benign and malignant.

The EDA indicated that the features have different numerical scales. Therefore, standardization was applied before algorithms that depend on feature magnitude and before PCA.

5 Data Preprocessing

The following preprocessing steps were used:

1. The diagnosis column was separated as the target variable.
2. The categorical target values were encoded as 0 and 1.
3. The data was divided into training and testing sets using an 80:20 split.
4. Stratified splitting was used to preserve the class distribution.
5. Numerical features were standardized using `StandardScaler`.
6. PCA was applied only in the With-PCA experimental setting.

For the PCA setting, the processing pipeline was structured as:

`StandardScaler` → PCA → Classifier

For the No-PCA setting:

`StandardScaler` → Classifier

Using the preprocessing operations inside the pipeline ensures that scaling and PCA are fitted using the appropriate training data during cross-validation, thereby avoiding data leakage.

6 Machine Learning Algorithm

Ten classification algorithms were considered in this experiment.

6.1 Support Vector Machine

SVM attempts to find a decision boundary that maximizes the margin between different classes. Linear and RBF kernels were considered during hyperparameter tuning.

6.2 Naïve Bayes

Gaussian Naïve Bayes was used for classification. The `var_smoothing` parameter was tuned to control numerical smoothing.

6.3 k-Nearest Neighbors

KNN classifies a sample based on the labels of its nearest training samples. The number of neighbors, weighting strategy, and distance metric were tuned.

6.4 Logistic Regression

Logistic Regression is a linear classification algorithm that estimates the probability of a sample belonging to a particular class. The regularization parameter and solver/penalty combinations were tuned.

6.5 Decision Tree

Decision Tree classification recursively divides the feature space using decision rules. Parameters such as criterion, maximum depth, minimum samples for splitting, and minimum samples per leaf were considered.

6.6 Random Forest

Random Forest combines multiple decision trees and aggregates their predictions. The number of trees, maximum depth, and minimum split size were tuned.

6.7 AdaBoost

AdaBoost combines multiple weak learners sequentially, giving greater importance to previously misclassified observations. The number of estimators and learning rate were tuned.

6.8 Gradient Boosting

Gradient Boosting builds an ensemble of weak learners sequentially. The number of estimators, learning rate, and tree depth were tuned.

6.9 XGBoost

XGBoost is a gradient-boosting ensemble algorithm. The number of estimators, maximum tree depth, and learning rate were tuned.

6.10 Stacking

Stacking combines predictions from multiple base learners and uses a meta-learner to produce the final classification. This allows different learning algorithms to contribute to the final prediction.

7 Experimental Setup

Python Version	3.13.13
Scikit-Learn Version	1.7.5
IDE	Jupyter Notebook
Operating System	Fedora 42
Hardware	Intel Core i5 (11th gen), 8GB RAM
Cross-Validation	Stratified 5-Fold Cross-Validation
Evaluation Metric	Accuracy and F1-score

The models were trained under both No-PCA and With-PCA conditions. Hyperparameter optimization was performed using `GridSearchCV`.

The F1-score was used as the primary GridSearch scoring metric:

$$F1 = 2 \frac{Precision \times Recall}{Precision + Recall}$$

This metric provides a balanced measure of precision and recall and is particularly useful for binary classification.

8 Hyperparameter Selection

Different hyperparameter grids were selected for each classifier. The grids were intentionally kept moderate so that meaningful parameter comparisons could be performed without unnecessary computational overhead.

Model	Hyperparameters Investigated
SVM	Kernel: linear/RBF; C : 0.1, 1, 10; Gamma: scale/auto for RBF
Naïve Bayes	<code>var_smoothing</code> : 10^{-11} , 10^{-9} , 10^{-7} , 10^{-5} , 10^{-3}
KNN	$n_neighbors$: 3, 5, 7, 9, 11; weights: uniform/distance; metric: Euclidean/Manhattan
Logistic Regression	C : 0.1, 1, 10; linear solver and penalty combinations
Decision Tree	criterion: Gini/Entropy; max depth: 3, 5, 10; minimum split/leaf parameters
Random Forest	$n_estimators$: 100, 200; max depth: None, 5, 10; minimum split parameters
AdaBoost	$n_estimators$: 50, 100, 200; learning rate: 0.01, 0.1, 1.0
Gradient Boosting	$n_estimators$: 50, 100; learning rate: 0.05, 0.1; max depth: 2, 3
XGBoost	$n_estimators$: 100, 200; max depth: 3, 5; learning rate: 0.05, 0.1
Stacking	Meta-learner regularization parameter C : 0.1, 1, 10

9 Results

The ten classification algorithms were trained and evaluated under two conditions: the original feature space without PCA and the PCA-reduced feature space. Hyperparameter tuning was performed using GridSearchCV with stratified 5-fold cross-validation, with F1-score used as the optimization metric.

9.1 PCA Variance Explained

The original dataset contained 30 numerical features. After standardization, PCA was applied with a target of retaining 95% of the variance.

Table 3: PCA Variance Explained

Parameter	Value
Original Features	30
Variance Target	95%
Selected Components	10
Explained Variance	95.2069%

Thus, the dimensionality was reduced from 30 features to 10 principal components while retaining approximately 95.21% of the variance.

9.2 Hyperparameter Tuning Results

The best hyperparameters obtained for each classifier are summarized below.

Table 4: Best Hyperparameters Selected by Grid-SearchCV

Model	No-PCA Parameters	With-PCA Parameters
SVM	$C = 10$, kernel=RBF, gamma=scale	$C = 0.1$, kernel=linear
Naïve Bayes	var_smoothing= 10^{-11}	var_smoothing= 10^{-11}
KNN	$k = 3$, weights=uniform, metric=Manhattan	$k = 3$, weights=uniform, metric=Euclidean
Logistic Regression	$C = 1$, penalty=L2, solver=liblinear	$C = 0.1$, penalty=L2, solver=liblinear
Decision Tree	criterion=Entropy, max depth=10, min split=2, min leaf=2	criterion=Gini, max depth=5, min split=5, min leaf=1
Random Forest	100 estimators, max depth=5, min split=2	200 estimators, max depth=10, min split=2
AdaBoost	200 estimators, learning rate=1.0	100 estimators, learning rate=1.0
Gradient Boosting	100 estimators, learning rate=0.1, max depth=3	100 estimators, learning rate=0.1, max depth=2

XGBoost	200 estimators, learning rate=0.1, max depth=5	200 estimators, learning rate=0.1, max depth=5
Stacking	Final estimator $C = 10$	Final estimator $C = 10$

9.3 Cross-Validation Performance

The best mean 5-fold cross-validation F1-score obtained by each classifier is shown below.

Table 5: 5-Fold Cross-Validation F1-Score

Model	No-PCA CV F1	With-PCA CV F1
SVM	0.9674	0.9635
Naïve Bayes	0.9170	0.8824
KNN	0.9667	0.9605
Logistic Regression	0.9640	0.9701
Decision Tree	0.9042	0.9047
Random Forest	0.9553	0.9398
AdaBoost	0.9610	0.9605
Gradient Boosting	0.9553	0.9339
XGBoost	0.9615	0.9642
Stacking	0.9704	0.9764

9.4 Test-Set Performance

The best estimator selected by GridSearchCV was evaluated on the held-out test set.

Table 6: Test-Set Performance

Model	Setting	Accuracy	F1-Score
SVM	No PCA	0.9737	0.9630
SVM	With PCA	0.9825	0.9756
Naïve Bayes	No PCA	0.9211	0.8889
Naïve Bayes	With PCA	0.8947	0.8571
KNN	No PCA	0.9649	0.9500
KNN	With PCA	0.9474	0.9250
Logistic Regression	No PCA	0.9737	0.9639
Logistic Regression	With PCA	0.9825	0.9756
Decision Tree	No PCA	0.9561	0.9367
Decision Tree	With PCA	0.9386	0.9136
Random Forest	No PCA	0.9737	0.9630
Random Forest	With PCA	0.9474	0.9268
AdaBoost	No PCA	0.9737	0.9630
AdaBoost	With PCA	0.9649	0.9512
Gradient Boosting	No PCA	0.9649	0.9500
Gradient Boosting	With PCA	0.9561	0.9383
XGBoost	No PCA	0.9737	0.9630
XGBoost	With PCA	0.9737	0.9639

Stacking	No PCA	0.9912	0.9880
Stacking	With PCA	0.9737	0.9630

10 Result Visualization

The following visualizations were generated or considered for evaluating the classification models:

1. PCA cumulative explained variance plot.
2. Confusion matrix for selected best-performing models.
3. ROC curves for selected models.
4. Precision-Recall curves for selected models.

10.1 PCA Explained Variance

PCA was used to transform the standardized feature space into a smaller set of principal components while retaining a high proportion of the original variance.

The cumulative explained variance plot was used to determine the number of components required to retain the selected variance target.

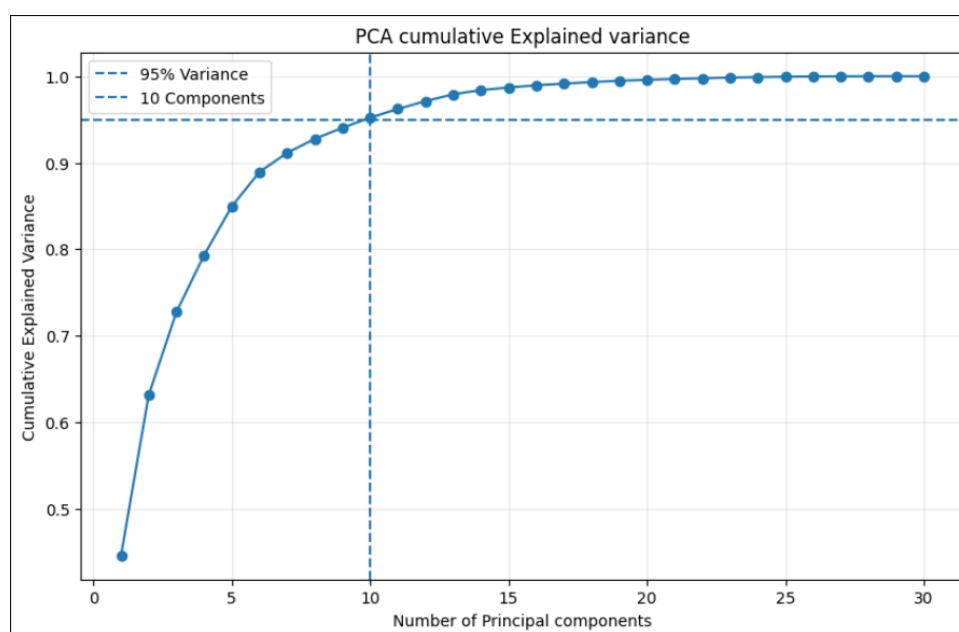


Figure 1: Cumulative explained variance of PCA components.

10.2 Confusion Matrix

A confusion matrix provides the number of true positives, true negatives, false positives, and false negatives produced by a classifier.

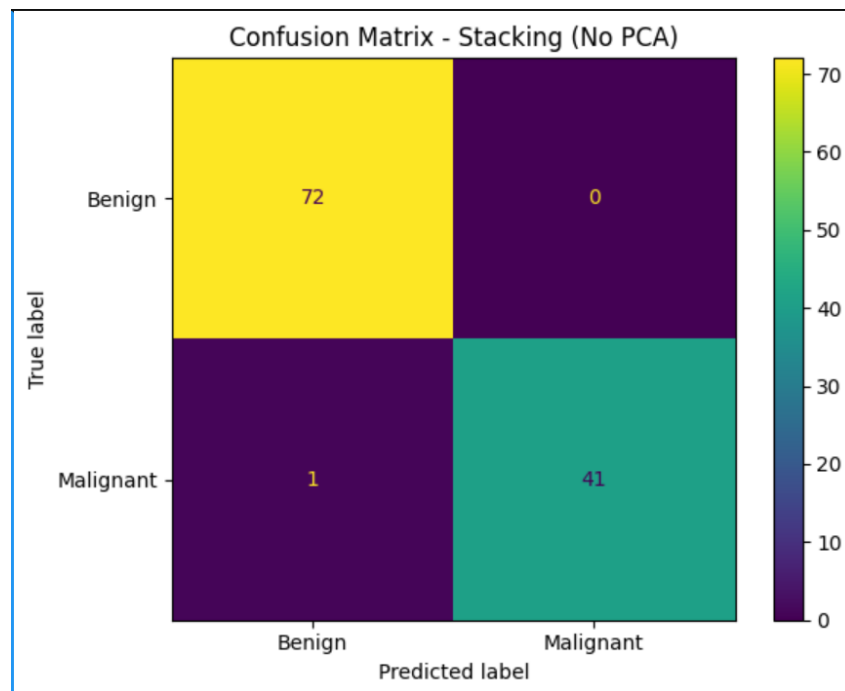


Figure 2: Confusion matrix of the selected best-performing classifier.

10.3 ROC Curve

The Receiver Operating Characteristic (ROC) curve illustrates the trade-off between the true-positive rate and false-positive rate at different classification thresholds.

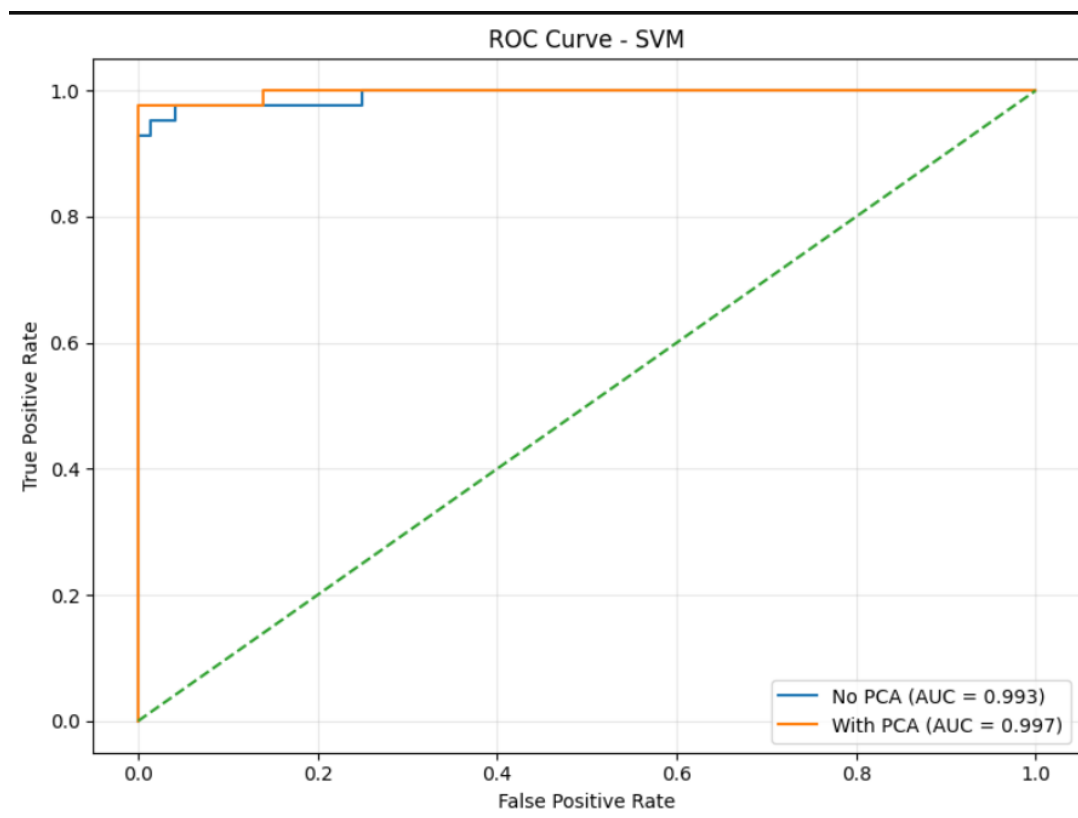


Figure 3: ROC curve of the selected classifiers.

10.4 Precision-Recall Curve

The Precision-Recall curve shows the relationship between precision and recall over different classification thresholds.

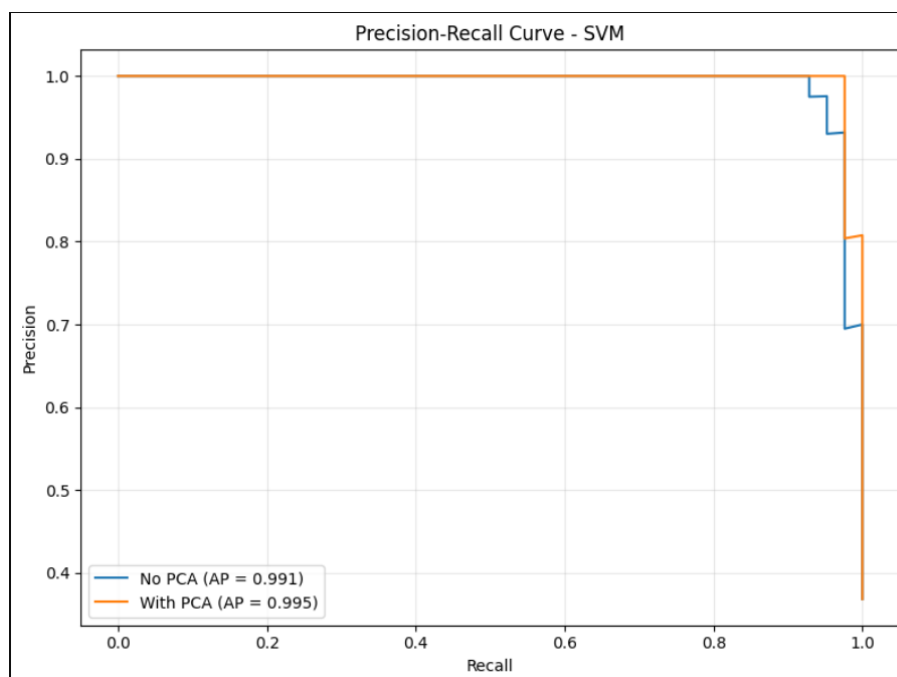


Figure 4: Precision-Recall curve of the selected classifiers.

11 Discussion

The results show that PCA had a model-dependent effect on classification performance. Logistic Regression, XGBoost, Decision Tree, and Stacking showed improved CV F1-scores with PCA. SVM and Logistic Regression also achieved better test-set performance after PCA. Naïve Bayes, KNN, Random Forest, AdaBoost, and Gradient Boosting performed better without PCA. The best overall test performance was obtained by Stacking without PCA, with 99.12% accuracy and 0.9880 F1-score. PCA reduced the feature space from 30 to 10 components while retaining 95.2069% variance. This shows that dimensionality reduction can reduce feature complexity without necessarily improving every model.

12 Conclusion

PCA successfully reduced the dimensionality of the dataset while preserving most of its variance. Its effect varied across the ten classification algorithms. Linear models such as SVM and Logistic Regression benefited from the reduced feature space in the test results. Several tree-based and distance-based models performed better using the original features. Stacking without PCA achieved the highest overall test performance. Therefore, PCA should be selected based on the model and experimental results rather than applied universally.

13 References

1. Scikit-learn Documentation, <https://scikit-learn.org/>
2. Pedregosa, F. et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, 12, 2825–2830, 2011.
3. UCI Machine Learning Repository, Breast Cancer Wisconsin (Diagnostic) Dataset.
4. Chen, T. and Guestrin, C., “XGBoost: A Scalable Tree Boosting System,” *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.